

CAHIER DES CHARGES

Outil de migration et d'audit de configurations firewall

Version : 1.0

IUT PTUT - Mars 2026

Migration bidirectionnelle WatchGuard Fireware <> pfSense

Document CONFIDENTIEL - Usage interne IUT PTUT

Sommaire

1. Contexte et enjeux

Les infrastructures réseau des entreprises modernes reposent sur une diversité croissante de constructeurs et de solutions de pare-feux. WatchGuard Fireware, avec ses firewalls Firebox, et pfSense, distribution BSD open-source, sont deux des solutions les plus répandues dans les PME/PMI et les environnements hybrides. Leur coexistence crée un besoin opérationnel fort : la capacité à transférer des politiques de sécurité d'un environnement à l'autre de manière fiable, rapide et sans risque d'erreur.

Une migration manuelle d'une configuration comportant plusieurs dizaines ou centaines de règles représente un investissement en temps considérable et introduit un risque élevé d'erreur humaine. Chaque règle mal transcrite peut créer une faille de sécurité ou bloquer du trafic légitime. Par ailleurs, l'hétérogénéité des formats XML propriétaires — deux représentations conceptuellement similaires mais syntaxiquement très différentes — rend la tâche particulièrement ardue sans outil dédié.

La mise en conformité avec les référentiels NIST SP 800-41, CIS Controls v8 et les recommandations de l'ANSSI impose également aux organisations de disposer d'outils d'audit automatisé capables d'identifier les mauvaises pratiques de configuration firewall en continu, de les quantifier et de proposer des actions correctives concrètes.

1.1 Constats terrain

- Absence d'outil libre de migration automatisée WatchGuard Fireware ↔ pfSense
- Risque de régression sécuritaire lors des migrations manuelles (règles oubliées, inversées)
- Hétérogénéité profonde des structures XML (alias, address-groups, nat, policies)
- Pas d'outil d'audit intégré avec scoring standardisé sur les deux plateformes
- Temps de migration estimé entre 2 et 10 jours pour une configuration d'entreprise

1.2 Objectifs stratégiques

- Réduire le temps de migration de plusieurs jours à quelques minutes par automatisation complète
- Garantir la fidélité sémantique : chaque règle source doit avoir un équivalent fonctionnel dans la cible
- Intégrer un audit de sécurité normé (NIST, CIS, ANSSI) pour détecter les vulnérabilités en temps réel
- Proposer un outil portatif (exe autonome) déployable sans infrastructure additionnelle
- Permettre l'auto-réparation des fichiers XML non conformes via intelligence artificielle

2. Architecture de NETMORPH

NETMORPH suit une architecture Modèle-Vue-Contrôleur rigoureuse avec une séparation nette des responsabilités entre les modules Python. L'interface utilisateur est rendue dans un composant QWebEngineView (Chromium intégré) qui communique avec le code Python via un protocole IPC basé sur un schéma d'URL propriétaire netmorph://.

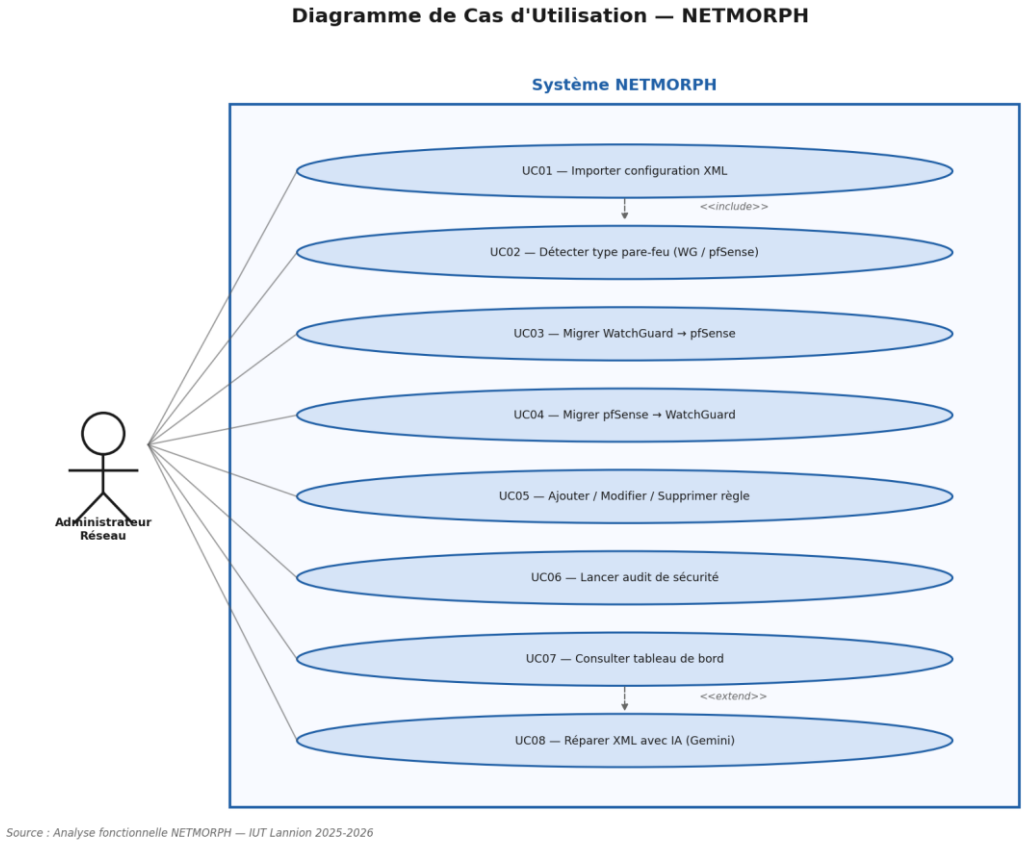


Figure 1 — Diagramme de cas d'utilisation NETMORPH : acteurs et fonctions principales

Le contrôleur (main.py) intercepte toutes les actions utilisateur via la classe CustomPage qui hérite de QWebEnginePage. Chaque clic sur un bouton de l'interface HTML génère une navigation vers une URL netmorph://action?params qui est parsée et routée vers la méthode Python correspondante. Cette approche évite l'utilisation de WebSockets ou de JavaScript bridges complexes tout en maintenant une séparation propre entre la présentation et la logique métier.

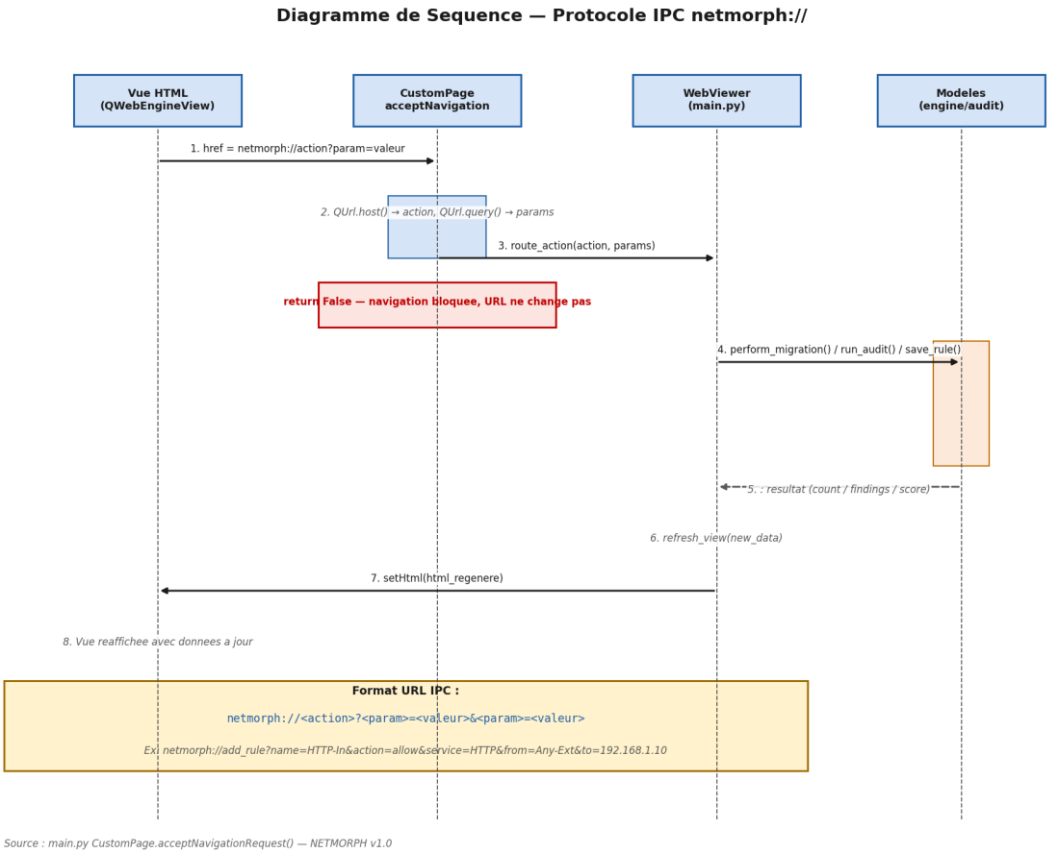


Figure 2 — Diagramme de séquence IPC netmorph:// : cycle complet d'une action utilisateur

3. Périmètre fonctionnel détaillé

3.1 Import et visualisation

L'import constitue la porte d'entrée de NETMORPH. L'utilisateur sélectionne un fichier XML via un dialogue natif. L'application détecte automatiquement le type de firewall en inspectant la balise XML racine : <profile> pour WatchGuard Fireware 12.x, <pfsense> pour pfSense 2.x. Un fichier avec une balise racine inconnue génère un avertissement explicite et ne perturbe pas le fonctionnement de l'application.

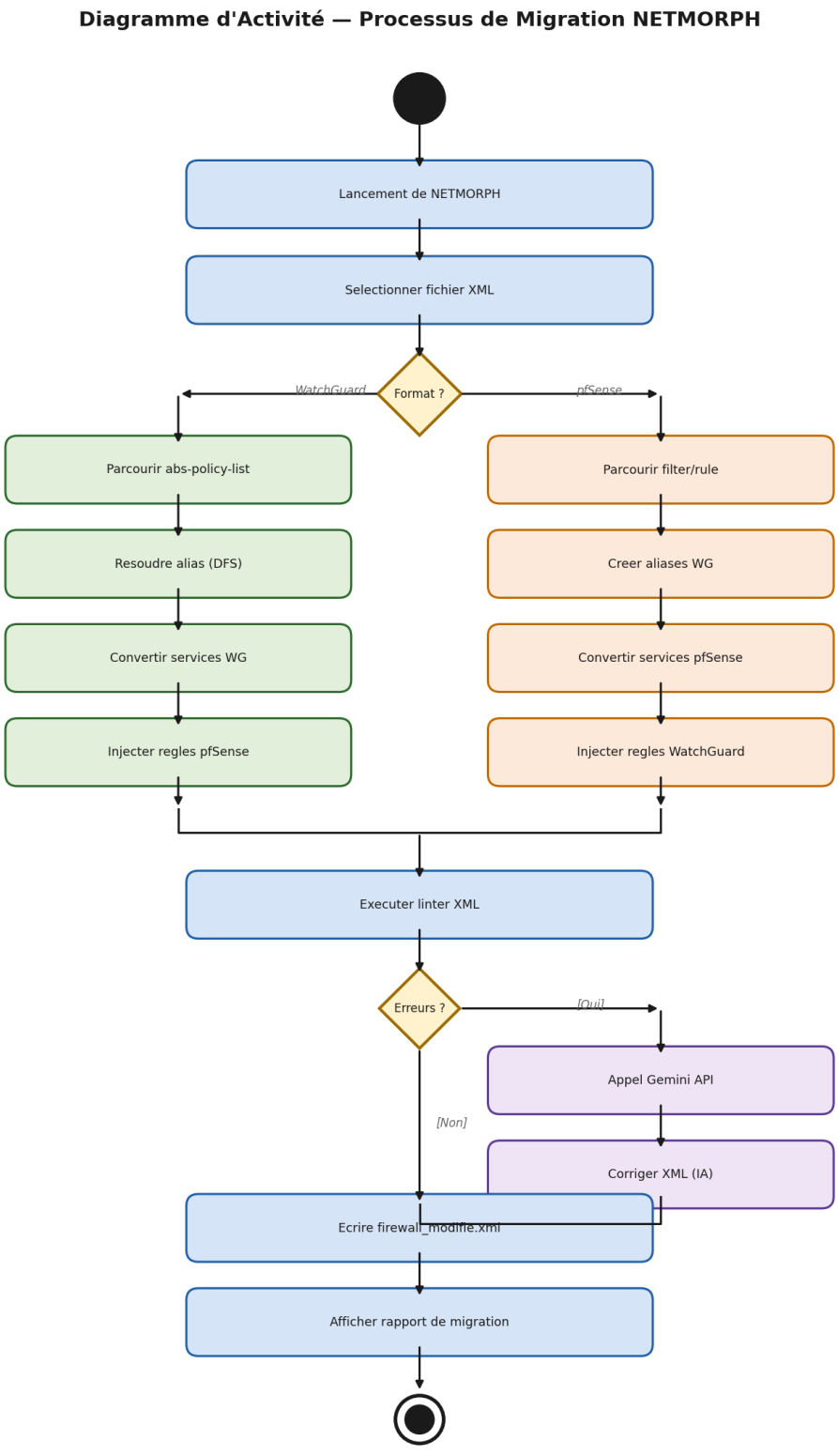
Après import réussi, le tableau de bord se remplit automatiquement avec toutes les informations extraites : informations système (hostname, modèle), interfaces réseau avec leurs adresses IP et noms de périphérique OS, règles de filtrage avec action codée en couleur (Allow en vert, Denied en rouge), services, sources, destinations et liaisons NAT. La logique d'affichage s'adapte au format : les alias WatchGuard sont résolus récursivement pour afficher les IPs réelles plutôt que les noms d'alias abstraits.

3.2 Migration bidirectionnelle

La migration est le cœur fonctionnel de NETMORPH. Elle transforme automatiquement une configuration complète d'un format vers l'autre, en préservant la sémantique de chaque règle : action (allow/deny), protocole, ports, sources et destinations. Le processus s'effectue en deux étapes guidées par un wizard :

1. Sélection du fichier source avec détection automatique du constructeur et affichage de la destination déduite
2. Sélection de la destination et lancement de la migration avec rapport détaillé

La migration WatchGuard → pfSense résout les alias en IPs réelles, mappe les services en protocole/port et convertit les règles SNAT (nat type=7) en règles nat/rule pfSense avec target et local-port. La migration pfSense → WatchGuard est plus complexe : elle crée les structures d'objets réseau obligatoires de WatchGuard (address-groups, alias wrappers) et construit les triples structures SNAT WatchGuard (address-group .1.snat + nat type=7 + alias .snat).



Source : engine.py — NETMORPH v1.0 — IUT Lannion

Figure 3 — Diagramme d'activité : flux complet du processus de migration NETMORPH

3.3 Audit de sécurité

L'onglet Audit de Sécurité analyse automatiquement la configuration active et produit un rapport complet basé sur 13 checks de sécurité couvrant les vulnérabilités les plus répandues dans les configurations firewall d'entreprise. Chaque finding est classé selon cinq niveaux de sévérité (CRITICAL, HIGH, MEDIUM, LOW, INFO) et contribue à un score global calculé sur 100 points par déduction de pénalités cumulatives.

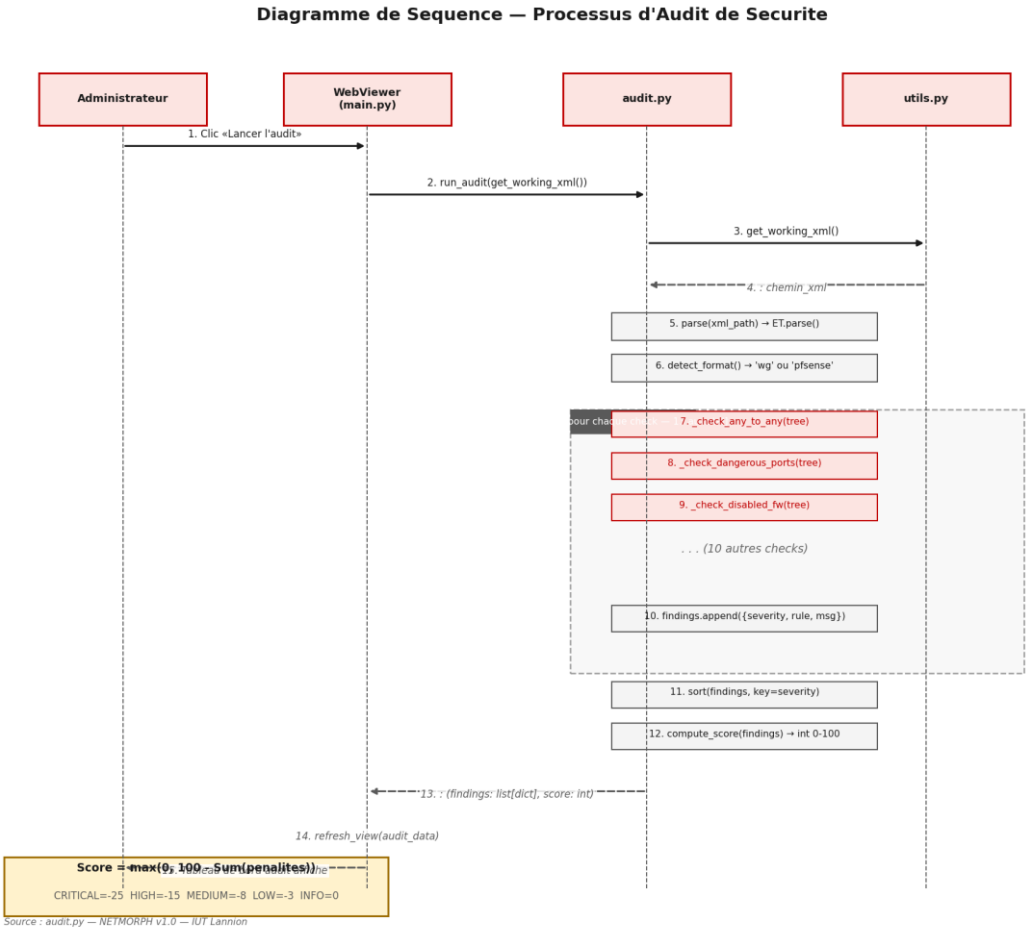


Figure 4 — Diagramme de séquence : processus d'audit de sécurité NETMORPH

Les checks s'appuient sur des référentiels de sécurité reconnus : le NIST SP 800-41 Rev 1 (Guidelines on Firewalls and Firewall Policy) pour les principes généraux, les CIS Controls v8 Control 12 (Network Infrastructure Management) pour les bonnes pratiques réseau, les recommandations ANSSI pour le filtrage et les CVE spécifiques tels que CVE-2019-0708 (BlueKeep — RDP) et MS17-010 (EternalBlue — SMB/445) pour les vulnérabilités critiques connues.

3.4 Auto-réparation XML par intelligence artificielle

NETMORPH intègre un mécanisme d'auto-réparation XML optionnel basé sur Gemini 1.5 Flash de Google. Avant toute écriture dans le fichier de travail, deux linters XML valident la conformité du XML généré avec les formats cibles. Le linter WatchGuard refuse les balises <source-port> et <source-port-enabled> dans les policies (incompatibles avec Fireware 12.x). Le linter pfSense exige la présence de <ipprotocol> dans chaque <rule>.

En cas d'échec du linter, si une clé API Gemini est configurée dans le fichier .env, le XML en erreur est envoyé à Gemini avec le message d'erreur précis pour correction automatique. L'IA retourne le XML corrigé qui est alors utilisé pour la persistance. Cette fonctionnalité est entièrement optionnelle et désactivée par défaut — l'application fonctionne normalement sans elle.

4. Contraintes techniques

NETMORPH est conçu pour fonctionner en environnement air-gap complet : aucune donnée de configuration firewall n'est transmise sur le réseau en dehors de l'appel API Gemini optionnel, qui nécessite une action explicite de configuration de la clé API par l'administrateur. Tous les traitements XML sont effectués localement en mémoire. Le fichier XML original importé (firewall.xml) n'est jamais modifié — toutes les opérations écrivent dans firewall_modifie.xml.

- Plateforme cible : Windows 10/11 64-bit (mode compilé), Linux Ubuntu 20.04+ (mode source)
- Langage : Python 3.9+, framework PyQt6 6.x, moteur web Chromium via QtWebEngine
- Format WatchGuard : Fireware 12.x (balise racine <profile>)
- Format pfSense : 2.4+ CE et Plus (balise racine <pfsense>)
- Packaging : PyInstaller — livrable unique Netmorph.exe sans dépendances
- Configuration IA : optionnelle, GEMINI_API_KEY dans .env gitignored

5. Installation et déploiement

5.1 Mode exécutable autonome (utilisateur final)

3. Télécharger Netmorph.exe depuis le dépôt de releases
4. Copier dans un dossier avec template/ (Template_Firebox.xml + Template_pfsense.xml)
5. Double-cliquer — aucune installation requise
6. Si SmartScreen bloque : clic droit > Propriétés > Débloquer

5.2 Mode développeur (sources Python)

```
git clone https://github.com/Ilyesse26100/NETMORPH.git
cd NETMORPH
python -m venv venv && venv\Scripts\activate
pip install -r requirements_fixed.txt
python main.py
```

5.3 Compilation vers .exe

```
compile_fixed.bat   # Windows
bash compile.sh     # Linux/Mac
# Resultat : dist/Netmorph.exe  (portable, ~300 Mo avec Qt + Chromium)
```

6. Feuille de route

La version 1.0 de NETMORPH couvre la migration bidirectionnelle WatchGuard Fireware ↔ pfSense avec audit de sécurité intégré. Les évolutions planifiées sont les suivantes :

6.1 Version 2.0

- Support Cisco ASA (.cfg) et Fortinet FortiGate (import/export)
- Mode comparaison diff entre deux configurations (avant/après migration)
- Export du rapport d'audit en PDF avec recommandations détaillées
- Historique des migrations avec possibilité de rollback vers une version précédente
- Mode ligne de commande (CLI) pour intégration dans des pipelines CI/CD

6.2 Version 3.0

- Interface web (mode serveur Flask/FastAPI) pour les équipes distribuées
- API REST pour l'intégration dans des plateformes SOAR et SIEM
- Connecteurs directs avec les API WatchGuard (WSM) et pfSense (pfSense API)
- Audit en temps réel via collecte SYSLOG (RFC 5424)

Annexe A — Correspondances XML WatchGuard ↔ pfSense

Concept réseau	WatchGuard Fireware XML	pfSense XML
Fichier racine	<profile>	<pfsense>
Règle filtrage	<abs-policy-list>/<abs-policy>	<filter>/<rule>
Action Allow	<firewall>1</firewall>	<type>pass</type>
Action Block	<firewall>0</firewall>	<type>block</type>
Règle système	<property>32</property> (cachée)	<disabled/> (ignorée)
Source any	<alias>Any</alias>/<alias>Any-External</alias>	<source><any/></source>
Dest. any	<alias>Any</alias>	<destination><any/></destination>
Interface WAN	Any-External	wan
Interface LAN	Any-Trusted	lan
IP hôte	address-group type=1 + alias	<address>192.168.1.10</address>
Réseau CIDR	address-group type=2 + ip-mask	<network>192.168.1.0/24</network>
Port forwarding	<nat> type=7 (SNAT)	<nat>/<rule> + target + local-port
Masquerade	<nat> type=3	(automatique pfSense)
VLAN	<interface>/<vlan-if> + tag	<vlans>/<vlan> + <tag>
Objet réseau	<alias-list>/<alias>	<aliases>/<alias>